

CONFIGURAÇÃO DE AMBIENTE

Lista de exercícios — Segredos, Config e Trabalho em Equipe

Continuação da lista de variáveis de ambiente em Node.js e TypeScript

Atividade prática

Esta lista contém quatro exercícios. Os códigos possuem falhas intencionais e os trechos de apoio não entregam a resolução completa. A diferença para a lista anterior é o alvo: aqui o defeito não impede a aplicação de subir. Ela sobe, funciona e só cobra o preço depois, quando a chave vaza, quando outro desenvolvedor clona o projeto ou quando o ambiente muda.

Exercícios

- 7 - Chaves de API e a diferença entre segredo no código e segredo no ambiente.
- 8 - Estrutura de projeto com um módulo de configuração.
- 9 - Boas práticas em equipe com arquivo de exemplo.
- 10 - Variáveis dinâmicas e comportamento por ambiente.

Em cada exercício, registre o sintoma, sua hipótese, a evidência encontrada e a correção aplicada. Aqui a evidência costuma ser aquilo que ficou visível para alguém que não deveria ver: um log, um commit, uma URL, um arquivo que foi parar no repositório.

Pré-requisitos

```
npm install express dotenv
npm install -D typescript ts-node @types/node @types/express

# usado a partir do exercício 10
npm install dotenv-expand

git init
```

Esta lista continua o projeto dos exercícios 1 a 5. O módulo de configuração criado no exercício 1 e o arquivo de exemplo do exercício 3 são retomados aqui.

Orientações de investigação

Estes defeitos não aparecem em um erro de execução. Para encontrá-los é preciso olhar o projeto do ponto de vista de quem está de fora: o repositório, o log e a máquina do colega.

Etapas	O que fazer
1. Reproduzir	Simular a situação real: um clone limpo, um deploy, um log sendo lido.
2. Observar	Procurar onde o valor sensível aparece e quem consegue vê-lo.
3. Formular hipótese	Apontar se o problema é o lugar do valor, o seu tipo ou a sua origem.
4. Investigar	Conferir repositório, saída da aplicação e caminho percorrido pelo valor.
5. Corrigir	Aplicar a menor mudança que resolva a causa.
6. Verificar	Repetir a partir de um clone limpo e em mais de um ambiente.

Registro obrigatório

Sintoma:
Como foi reproduzido:
Resultado esperado:
Resultado obtido (onde o valor apareceu, e para quem):
Hipótese:
Como foi investigado:
Causa encontrada:
Como a correção foi validada:

Regras

- Não escrever segredo no código, nem como valor padrão de uma variável ausente.
- Não usar chave real nos exercícios: gere um valor fictício e trate-o como se fosse real.
- Não imprimir, logar ou enviar a chave em URL, mensagem de erro ou resposta da API.
- Não ler process.env fora do módulo de configuração.
- Não deixar o arquivo de exemplo desatualizado em relação às variáveis realmente usadas.
- Não resolver conflito de ambiente apagando o arquivo local do colega.

Avaliação: o raciocínio usado para encontrar a falha vale tanto quanto o resultado final.

Exercício 7 - Chaves de API

A função consome uma API fictícia e funciona na máquina de quem a escreveu. A chave foi movida para o arquivo de ambiente, mas o código antigo continua ali como rede de segurança. Depois de um incidente, descobriu-se que a chave aparecia em três lugares diferentes.

```
// src/weather.ts
const API_KEY = process.env.API_KEY || 'sk_live_9f2a8c1d4b7e';

export async function getWeather(city: string) {
  const url = `https://api.exemplo.com/v1/weather?city=${city}&key=${API_KEY}`;

  console.log('chamando:', url);

  const response = await fetch(url);

  if (!response.ok) {
    throw new Error(`falha na API: ${response.status} - chave ${API_KEY}`);
  }

  return response.json();
}
```

Tarefas

- Listar os três pontos em que a chave fica exposta neste trecho.
- Explicar por que o valor padrão é pior que a ausência da variável.
- Remover o valor embutido e falhar na subida quando a chave não existir.
- Enviar a chave em cabeçalho de autorização, em vez de parâmetro da URL.
- Remover a chave do log e da mensagem de erro, mantendo o diagnóstico útil.
- Criar uma função de mascaramento para quando for necessário citar a chave.
- Comparar os dois cenários: trocar a chave com ela no código e com ela no ambiente.
- Descrever o que precisa acontecer quando uma chave é exposta por engano.

Pistas graduais

- Pense em quem consegue ler a URL completa de uma requisição depois que ela acontece.
- Verifique se a aplicação avisa de alguma forma quando cai no valor padrão.
- Investigue a diferença entre parâmetro de consulta e cabeçalho quanto ao registro em log.
- Observe que a mensagem de erro costuma ser enviada para o monitoramento.
- Considere quantos passos cada cenário exige para substituir a chave.

Exercício 7 - Apoio e verificação

Trecho de apoio

```
import { config } from './config'; // a chave vem daqui, já validada

function mask(secret: string): string {
  /* mostre apenas os últimos caracteres, o bastante para identificar */
}

export async function getWeather(city: string) {
  const url = new URL('https://api.exemplo.com/v1/weather');
  url.searchParams.set('city', city);

  const response = await fetch(url, {
    headers: { /* qual cabeçalho transporta a credencial? */ }
  });

  if (!response.ok) {
    /* registre status e corpo, nunca a credencial */
  }

  return response.json();
}
```

Comparação pedida

Pergunta	Chave no código	Chave no ambiente
Trocar a chave exige o quê?		
Quem consegue lê-la?		
Dois ambientes, duas chaves?		
O que acontece se vazar?		

Preencha a tabela com suas próprias palavras: ela é parte da entrega.

Cenários mínimos

Cenário	O que verificar
API_KEY definida	A chamada funciona e a chave não aparece em lugar nenhum.
API_KEY ausente	A aplicação falha na subida, sem usar valor embutido.
Resposta 401 da API	O erro é claro e não contém a credencial.
Log completo da execução	Nenhuma linha permite reconstruir a chave.

Perguntas orientadoras

- Por que um segredo em parâmetro de URL é mais arriscado que em cabeçalho?
- O que torna um valor padrão perigoso justamente por funcionar?
- Por que mascarar é diferente de esconder?

Exercício 8 - Estrutura de Projeto

O módulo de configuração foi criado, mas metade do projeto continua lendo `process.env` direto. A porta chega como texto em um arquivo e como número em outro, e um teste conseguiu alterar a configuração em tempo de execução sem que ninguém percebesse.

```
// src/config.ts
export const config = {
  port: process.env.PORT,
  db: { host: process.env.DB_HOST }
};
```

```
// src/server.ts
import { config } from './config';
import dotenv from 'dotenv';

dotenv.config();

app.listen(config.port);

// src/db.ts
export const connection = connect(process.env.DB_HOST + ':5432');

// src/admin.ts
import { config } from './config';
config.db.host = 'localhost'; // ninguém impediu
```

Tarefas

- Imprimir o tipo de `config.port` e explicar de onde vem a inconsistência.
- Explicar por que `config.port` chega vazio mesmo com `PORT` definida no arquivo.
- Carregar o ambiente antes de qualquer `import` que dependa dele.
- Converter e validar os valores dentro do próprio módulo de configuração.
- Tipar o objeto de forma que `host` e `port` nunca sejam opcionais no restante do código.
- Impedir que a configuração seja alterada em tempo de execução.
- Substituir as leituras diretas de `process.env` em `db.ts` pelo módulo.
- Agrupar as variáveis por área e documentar cada campo exportado.

Pistas graduais

- Lembre que o `import` de `config.ts` é resolvido antes da primeira linha de `server.ts`.
- Verifique o tipo que o TypeScript infere para qualquer leitura de `process.env`.
- Procure o que acontece quando um objeto exportado é mutável e compartilhado.
- Estude a diferença entre congelar o objeto e apenas declará-lo como constante.
- Pense no que muda quando a conversão acontece uma vez, na borda do sistema.

Exercício 8 - Apoio e verificação

Trecho de apoio

```
import 'dotenv/config'; // primeira linha do módulo, antes de qualquer leitura

function requiredString(name: string): string {
  /* lance erro claro quando ausente ou vazio */
}

function requiredPort(name: string, fallback: number): number {
  /* converta, valide a faixa e recuse entrada inválida */
}

export const config = Object.freeze({
  env: /* ... */,
  port: requiredPort('PORT', 3000),
  db: Object.freeze({ host: requiredString('DB_HOST') })
});

export type Config = typeof config;
```

Cenários mínimos

Cenário	O que verificar
Aplicação iniciada	config.port é número e config.db.host é texto preenchido.
Tentativa de alterar config	A alteração falha ou é ignorada, e isso é demonstrado.
Busca por process.env no projeto	Só aparece dentro do módulo de configuração.
DB_HOST ausente	Erro na subida apontando a variável que faltou.
Uso do tipo exportado	O editor não oferece mais o caso undefined.

Perguntas orientadoras

- Por que process.env é sempre tipado como possivelmente indefinido?
- Qual a diferença entre uma constante e um objeto congelado?
- Que problema surge quando duas partes do código convertem o mesmo valor?
- Por que a conversão deve acontecer na borda, e não no ponto de uso?
- Que vantagem o tipo exportado traz para quem consome a configuração?

Critério: uma busca por process.env no projeto deve retornar ocorrências apenas no módulo de configuração.

Exercício 9 - Boas Práticas em Equipe

Três pessoas trabalham no mesmo projeto. Uma delas adicionou uma variável nova e a aplicação parou de subir na máquina das outras duas. No meio da confusão, alguém commitou o próprio arquivo de ambiente para resolver o problema.

```
$ git log --oneline -3
a91c20f adiciona integração de pagamento
7d3e881 corrige porta local
4b55e2c sobe .env para o time conseguir rodar <-- problema

$ git pull
Auto-merging .env.example
CONFLICT (content): Merge conflict in .env.example

$ npm run start:dev
Error: variável PAYMENT_URL ausente
```

```
# .env.example (desatualizado)
PORT=3000
DB_HOST=localhost
API_KEY=

# .env da pessoa que criou a integração (não versionado)
PORT=3000
DB_HOST=localhost
API_KEY=sk_test_abc
PAYMENT_URL=https://sandbox.pagamento.exemplo
```

Tarefas

- Explicar por que a aplicação quebrou nas outras máquinas, e não na de quem criou a variável.
- Desfazer o commit que subiu o arquivo de ambiente e tratar a chave exposta.
- Resolver o conflito do arquivo de exemplo mantendo as chaves das duas pessoas.
- Escrever a regra de equipe: toda variável nova entra no exemplo no mesmo commit.
- Criar um script que compara as chaves do .env com as do .env.example.
- Fazer esse script rodar antes da aplicação subir e apontar o que falta.
- Documentar no README os passos para um clone limpo começar a rodar.
- Explicar por que o exemplo deve ter nomes e nunca valores reais.

Pistas graduais

- Pense no que existe na máquina de quem criou a variável e não existe nas outras.
- Procure por que um arquivo de ambiente gera conflito com tanta facilidade.
- Considere que um commit removido continua acessível no histórico.
- Veja que comparar as chaves dos dois arquivos é uma operação simples de texto.
- Pense em quando o aviso é mais útil: na subida ou no meio da primeira requisição.

Exercício 9 - Apoio e verificação

Trecho de apoio

```
// scripts/check-env.ts - roda antes da aplicação
import fs from 'node:fs';

function keys(file: string): string[] {
  return fs.readFileSync(file, 'utf-8')
    .split('\n')
    /* descarte comentários e linhas vazias */
    /* fique apenas com o nome antes do sinal de igual */;
}

const missing = /* chaves do exemplo que não existem no .env local */;

if (missing.length > 0) {
  /* liste o que falta e interrompa com código diferente de zero */
}
```

```
# package.json
"scripts": {
  "predev": "ts-node scripts/check-env.ts",
  "start:dev": "cross-env NODE_ENV=development ts-node src/app.ts"
}
```

Cenários mínimos

Cenário	O que verificar
Clone limpo do repositório	O README leva a aplicação a subir sem perguntar nada ao time.
Variável nova no exemplo	A verificação aponta a falta antes da aplicação subir.
Variável extra só no .env local	Não quebra: o exemplo é o mínimo, não o máximo.
Conflito no arquivo de exemplo	Resolvido mantendo as chaves de ambos os lados.
Arquivo de ambiente no git status	Não aparece mais, em nenhuma máquina.

Perguntas orientadoras

- Por que o arquivo de exemplo deve ser versionado e o de ambiente não?
- O que torna um arquivo de ambiente especialmente sujeito a conflito?
- Por que a verificação compara nomes de chaves e nunca valores?
- Como um novo integrante descobre o que precisa configurar?
- Por que remover o commit não basta quando um segredo foi versionado?

Entrega: além do código, anexe o README e a saída da verificação com uma variável faltando.

Exercício 10 - Variáveis Dinâmicas

A ideia era que o caminho do log mudasse sozinho conforme o ambiente. O arquivo foi escrito com a sintaxe de interpolação, mas o programa criou uma pasta com um nome literal estranho e depois passou a falhar ao escrever.

```
# .env
NODE_ENV=development
LOG_PATH="./logs/${NODE_ENV}.log"
LOG_LEVEL=debug
```

```
// src/logger.ts
import fs from 'node:fs';
import dotenv from 'dotenv';

dotenv.config();

const logPath = process.env.LOG_PATH;

export function log(message: string) {
  fs.appendFileSync(logPath, `${new Date()} ${message}\n`);
}

log('aplicação iniciada');
```

```
$ ls logs/
'${NODE_ENV}.log'

$ NODE_ENV=production npm start
Error: ENOENT: no such file or directory, open './logs/${NODE_ENV}.log'
```

Tarefas

- Explicar por que o nome do arquivo saiu literal, com cifrão e chaves.
- Fazer a interpolação funcionar e comprovar com os dois ambientes.
- Explicar o que acontece com o caminho quando NODE_ENV não está definida.
- Garantir que a pasta de destino exista antes da primeira escrita.
- Tratar o caminho como texto possivelmente indefinido ao tipar a variável.
- Ajustar o nível de log por ambiente, usando a mesma lógica.
- Impedir que um valor externo aponte o log para fora da pasta do projeto.
- Mover a leitura dessas variáveis para o módulo de configuração do exercício 8.

Exercício 10 - Apoio e investigação

Trecho de apoio

```
import path from 'node:path';
import fs from 'node:fs';
import dotenv from 'dotenv';
import dotenvExpand from 'dotenv-expand';

const parsed = dotenv.config();
dotenvExpand.expand(parsed); // a expansão vem depois da leitura

const env = process.env.NODE_ENV ?? /* qual padrão? */;
const logPath = path.resolve(/* ancore na raiz do projeto */, process.env.LOG_PATH ?? '');

/* confirme que logPath continua dentro da pasta esperada */
fs.mkdirSync(path.dirname(logPath), { recursive: true });
```

Pistas graduais

- Descubra se o dotenv, sozinho, substitui referências a outras variáveis.
- Verifique o que as aspas ao redor do valor fazem com a expansão.
- Observe em que ordem a leitura e a expansão precisam acontecer.
- Teste o resultado quando a variável referenciada não existe.
- Compare o caminho resolvido com a raiz do projeto para detectar uma saída indevida.

Comportamento por ambiente

Variável	development	production
LOG_PATH	./logs/development.log	./logs/production.log
LOG_LEVEL	debug	info
Formato do log	legível para pessoa	estruturado para máquina
Stack em resposta de erro	pode aparecer	nunca

Use esta tabela como alvo: a aplicação deve chegar nesses valores sem nenhum if espalhado pelo código, apenas pela configuração.

Exercício 10 - Verificação e entrega

Cenários mínimos

Execução	O que verificar
NODE_ENV=development	Escreve em logs/development.log, com nível debug.
NODE_ENV=production	Escreve em logs/production.log, com nível info.
Sem NODE_ENV	Padrão definido, sem arquivo chamado undefined.
Pasta logs inexistente	Criada automaticamente antes da primeira escrita.
LOG_PATH apontando para fora	Recusado com erro claro na subida.

Perguntas para responder

- Por que a expansão de variáveis não é comportamento padrão do dotenv?
- Qual a diferença entre a interpolação do shell e a do arquivo de ambiente?
- Por que ancorar o caminho na raiz do projeto é mais seguro que usar caminho relativo?
- Que risco existe em montar um caminho de arquivo a partir de valor externo?
- Por que ajustar comportamento por configuração é melhor que por condicional no código?

Checklist final

Exercício	Entrega mínima
7	Chave apenas no ambiente, enviada em cabeçalho, ausente do log e do erro, tabela comparativa preenchida.
8	Configuração validada e congelada, tipos sem undefined e process.env apenas no módulo de configuração.
9	Arquivo de ambiente fora do histórico, exemplo atualizado, script de verificação e README para clone limpo.
10	Interpolação funcionando, pasta criada, padrão sem NODE_ENV definido e comportamento por ambiente comprovado.

Entrega final: envie o projeto com o módulo de configuração, o arquivo de exemplo atualizado, o script de verificação e um pequeno registro explicando, por exercício, qual era a causa da falha e como ela foi comprovada.